

Verificación de programas I

Algoritmos y estructuras de datos

by Damy



Tripla de Hoare

Cuando un programa S es correcto respecto a la especificación (P, Q) , lo denotamos con la siguiente tripla de Hoare:

$$\{P\} S \{Q\}$$

Precondición más débil (WP)

Dadas Q y S .

¿Qué es la $WP(S, Q)$?

Es la P más débil tal que $\{P\} S \{Q\}$





Dados P , S y Q cualquiera

¿Cómo podemos estar seguros que la tripla de Hoare:

$$\{P\} S \{Q\}$$

Es formalmente valida?

- Calculamos la $WP(S, Q)$
 - Si $P \rightarrow WP(S, Q)$, eso significa que la tripla es valida, pues $WP(S, Q)$ es la más débil de todas tal que hace valer la tripla, y P logro implicarla.
- 

SmallLang: Sintaxis

Es el lenguaje que usaremos para escribir S

- **Asignación:** $x := E$
- **Nada:** **Skip**
- **Secuencia:** $S1; S2$ es un programa, si $S1$ y $S2$ son programas
- **Condicional:** **if B then S1 else S2 endif** es un programa, si B es una expresión lógica y $S1$ y $S2$ son dos programas
- **Ciclo:** **While B do S endwhile** es un programa, si B es una expresión lógica y S es un programa

SmallLang: Ejemplos

$x := x + 2;$

Skip;

If($x == 3$) then

$y := 5$

Else

$y := s[3] + 2$

Endif;

Skip

$i := 0;$

While($i < x$) do

$y := y + 10;$

$i := i + 1$

Endwhile;

If($y < 0$) then

$y := 0$

Else

Skip

endif

SmallLang: Ejemplos

Observación: El programa de la izquierda puede ser escrito de la siguiente manera:

```
x := x + 2; Skip; If(x == 3) then y := 5 Else y := s[3] + 2 endif; Skip
```

Esto lo hará más sencillo a la hora de calcular la WP cuando apliquemos el "Axioma 3" de las diapositivas posteriores.

Predicados útiles

- ❑ Dada una expresión E , $\text{def}(E)$ son las condiciones necesarias para que E este definida.
- ❑ Dado un predicado Q , el predicado Q_E^x se obtiene reemplazando todas las apariciones libres de la variable x por E

Ejemplos

Considerando que: $x, z, y: \mathbb{Z}$, $S: \text{seq}\langle \mathbb{Z} \rangle$:

- ❑ $\text{Def}(x) \equiv \text{True}$ (Asumimos que las variables siempre están definidas)
- ❑ $\text{Def}(S[i]) \equiv 0 \leq i < |S|$
- ❑ $\text{Def}(S[0]) \equiv |S| \geq 1$
- ❑ $\text{Def}(1/y) \equiv y \neq 0$

Si $Q \equiv z+4$:

- ❑ $Q_y^z \equiv y+4$

Axiomas

- ❖ **Axioma 1:** $wp(x := E, Q) \equiv \text{def}(E) \wedge_L Q_E^x$
- ❖ **Axioma 2:** $wp(\text{skip}, Q) \equiv Q$
- ❖ **Axioma 3:** $wp(S1;S2, Q) \equiv wp(S1, wp(S2, Q))$
- ❖ **Axioma 4:** Si **S = if B then S1 else S2 endif** entonces
$$wp(S, Q) \equiv \text{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q)))$$

Ejercicios

- a) Explicar cual seria la wp con sus palabras
- b) Calcular la wp a partir de los axiomas

Asumimos que, para lo que sigue:

$$x, y, z \in \mathbb{R} \mid i, a, b, c \in \mathbb{Z} \mid L: \text{seq}\langle \mathbb{Z} \rangle$$



(1) $\text{wp}(x := 1/a; y := z, y \neq 1/3)$

a) ...

b) ...





(1) $wp(x := 1/a; y := z, y \neq 1/3)$

¿Cómo razono el a)?

Claramente “a” tendría que ser distinto de 0 para que el programa no se indefina.
Luego, a “y” se le asigna el valor que tiene “z”, y Q pide que “y” debe ser distinto de 1/3,
por lo que “z” debe ser distinto de 1/3

a) “a” debe ser distinto de 0 y “z” debe ser distinto de 1/3



b) $wp(x := 1/a; y := z, y \neq 1/3)$

$Ax(3) \equiv wp(x := 1/a, wp(y := z, y \neq 1/3))$

$Ax(1) \equiv wp(x := 1/a, \text{def}(z) \wedge_L z \neq 1/3)$

$\equiv wp(x := 1/a, \text{true} \wedge z \neq 1/3)$

$\equiv wp(x := 1/a, z \neq 1/3)$

$Ax(1) \equiv \text{def}(1/a) \wedge_L z \neq 1/3$

$\equiv a \neq 0 \wedge z \neq 1/3$

Que es lo que habíamos dicho en a)



(2) $\text{wp}(x := x+2; x := y, x > 2)$

a) ...

b) ...



(2) $wp(x := x+2; x := y, x > 2)$

¿Cómo razono el a)?

A pesar de que el valor de “x” es modificado incrementándolo en 2, luego su valor es “pisado” por “y”, por lo que solamente pediríamos que “y” sea mayor que 2, es lo más débil posible para cumplir Q

a) “y” debe ser mayor que 2

b) $\text{wp}(x := x+2; x := y, x > 2)$

$\text{Ax}(3) \equiv \text{wp}(x := x+2, \text{wp}(x := y, x > 2))$

$\text{Ax}(1) \equiv \text{wp}(x := x+2, \text{def}(y) \wedge_L y > 2)$

$\equiv \text{wp}(x := x+2, \text{true} \wedge y > 2)$

$\equiv \text{wp}(x := x+2, y > 2)$

$\text{Ax}(1) \equiv \text{def}(x+2) \wedge_L y > 2$

$\equiv \text{true} \wedge y > 2$

$\equiv y > 2$

Que es lo que habíamos dicho en a)



(3) $wp(x := L[i]; y := x+2; \text{Skip}, y < 0)$

a) ...

b) ...



(3) wp($x := L[i]; y := x+2; \text{Skip}, y < 0$)

¿Cómo razono el **a)**?

El valor de “y” termina siendo “x” aumentado en 2, sin embargo anteriormente a “x” se le asigna lo que solía haber en “L[i]”. Por lo que, lo que había en esa posición de “L” tiene que ser algo que aumentándole 2 sea menor que 0. Noto que el Skip no me modifica el programa entonces no lo tengo en cuenta, finalmente veo que “i” debe estar en el rango de la secuencia, para que al indexar, no se indefina.

a) i debe estar en rango y $L[i]+2$ debe ser menor que 0 (Es decir, $L[i]$ debe ser menor que -2)

b) $wp(x := L[i]; y := x+2; \text{Skip}, y < 0)$

$Ax(3) \equiv wp(x := L[i]; y := x+2, wp(\text{Skip}, y < 0))$

$Ax(2) \equiv wp(x := L[i]; y := x+2, y < 0)$

$Ax(3) \equiv wp(x := L[i], wp(y := x+2, y < 0))$

$Ax(1) \equiv wp(x := L[i], \text{def}(x+2) \wedge_L x+2 < 0)$

$\equiv wp(x := L[i], \text{True} \wedge_L x < -2)$

$Ax(1) \equiv \text{def}(L[i]) \wedge_L L[i] < -2$

$\equiv 0 \leq i < |L| \wedge_L L[i] < -2$

Que es lo que habíamos dicho en a)



(4) $wp(L[i+1] := a, (\forall j: \mathbb{Z})(0 \leq j < |L| \rightarrow (L[j] > 0 \wedge L[j] \bmod 2 = 0)))$

a)...

b)...



$$(4) \text{wp}(L[i+1] := a, (\forall j: \mathbb{Z})(0 \leq j < |L| \rightarrow (L[j] > 0 \wedge L[j] \bmod 2 = 0)))$$

¿Cómo razono el a)?

Según "Q", todos elementos de L deben ser positivos y pares. Entonces todos los valores que no modifiqué de L, es decir, aquellos que no estén en la posición "i+1" deberían de por si ser positivos y pares. Luego el valor que se le asigna en la posición "i+1", ósea "a", debe ser positivo y par. Finalmente, "i+1" debe estar entre 0 y la |L| para que no se indefina

a) "i+1" debe estar en rango de "L", todos los elementos en las posiciones distintas a "i+1" deben ser positivos y pares, y "a" debe ser positivo y par

(4) $wp(L[i+1] := a, (\forall j: \mathbb{Z})(0 \leq j < |L| \rightarrow (L[j] > 0 \wedge L[j] \bmod 2 = 0)))$

Wait

¿Puedo aplicar el axioma 1 con $L[i+1] := a$?

Nop

¿Cómo podemos reescribirlo
para aplicarlo?



setAt al rescate

$\text{setAt}(L, i, E)$ = Devuelve una secuencia igual a la original, pero con el elemento en la posición $L[i]$ cambiado por E

Puedo reescribir $L[i] := E$ como $L := \text{setAt}(L, i, E)$ y así aplicar el axioma 1

Obs: $\text{def}(\text{setAt}(L, i, E)) \equiv$
 $(\text{def}(L) \wedge \text{def}(i) \wedge \text{def}(E)) \wedge_L (0 \leq i < |L|)$



Observaciones importantes

I. $\text{def}(\text{setAt}(L, i, E)) \equiv (\text{def}(L) \wedge \text{def}(i) \wedge \text{def}(E)) \wedge_L (0 \leq i < |L|)$

II. $|\text{setAt}(L, i, E)| = |L|$

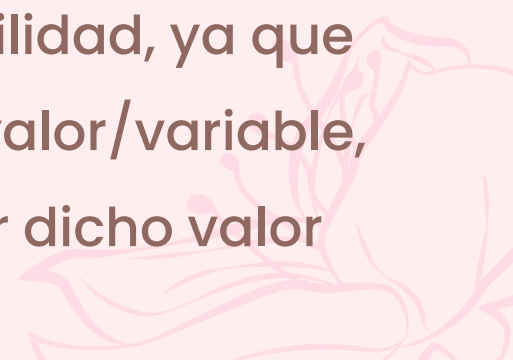
III. $0 \leq i < |L| \wedge_L (\forall j: \mathbb{Z}) ((0 \leq j < |L| \wedge j \neq i) \rightarrow_L \text{setAt}(L, i, E)[j] = L[j])$

IV. $0 \leq i < |L| \wedge_L \text{setAt}(L, i, E)[i] = E$

V. $0 \leq k < |L| \wedge_L (\forall j: \mathbb{Z}) ((0 \leq j < |L| \wedge j = k) \rightarrow_L S[j] = 2)$ es lo mismo
que: $0 \leq k < |L| \wedge_L S[k] = 2$



Observaciones importantes En español

- I. Para que $\text{setAt}(L, i, E)$ este definido, deben estar definidos todos sus parámetros, e " i " debe estar en rango
 - II. $\text{setAt}(L, i, E)$ solo cambia un valor de L , el tamaño se preserva
 - III. Todo valor en L distinto de i , sigue siendo igual al que había antes
 - IV. El $\text{setAt}(L, i, E)$ cambia lo que había en la posición i por " E ", entonces al indexarlo en " i " será igual a " E "
 - v. Si tengo un $(\forall j: \mathbb{Z})((j = \text{"valor"}) \rightarrow_L \text{"algo"})$, el $\forall$ deja de tener utilidad, ya que en el antecedente ya estas fijando a que " j " sea igual a cierto valor/variable, podrías escribir directamente ese " algo " reemplazando " j " por dicho valor
- 

$$\text{b) } \text{wp}(L[i+1] := a, (\forall j: \mathbb{Z})(0 \leq j < |L| \rightarrow_L (L[j] > 0 \wedge L[j] \bmod 2 = 0)))$$

Reescribo

$$\equiv \text{wp}(L := \text{setAt}(L, i+1, a), (\forall j: \mathbb{Z})(0 \leq j < |L| \rightarrow_L (L[j] > 0 \wedge L[j] \bmod 2 = 0)))$$

Ax (1)

$$\equiv \text{def}(\text{setAt}(L, i+1, a)) \wedge_L (\forall j: \mathbb{Z})(0 \leq j < |\text{setAt}(L, i+1, a)| \rightarrow_L \\ (\text{setAt}(L, i+1, a)[j] > 0 \wedge \text{setAt}(L, i+1, a)[j] \bmod 2 = 0))$$

Obs (1)

$$\equiv (\text{def}(L) \wedge \text{def}(i+1) \wedge \text{def}(a)) \wedge_L 0 \leq i+1 < |L| \wedge_L (\forall j: \mathbb{Z})(0 \leq j < |\text{setAt}(L, i+1, a)| \\ \rightarrow_L (\text{setAt}(L, i+1, a)[j] > 0 \wedge \text{setAt}(L, i+1, a)[j] \bmod 2 = 0))$$

$$\equiv \text{True} \wedge_L -1 \leq i < |L|-1 \wedge_L (\forall j: \mathbb{Z}) (0 \leq j < |\text{setAt}(L, i+1, a)| \rightarrow_L (\text{setAt}(L, i+1, a)[j] > 0 \wedge \text{setAt}(L, i+1, a)[j] \bmod 2 = 0))$$

Obs (II)

$$\equiv -1 \leq i < |L|-1 \wedge_L (\forall j: \mathbb{Z}) (0 \leq j < |L| \rightarrow_L (\text{setAt}(L, i+1, a)[j] > 0 \wedge \text{setAt}(L, i+1, a)[j] \bmod 2 = 0))$$

Queremos **deshacernos** de los setAt

Podemos aprovechar que dicha función devuelve una secuencia exactamente igual a L, salvo una posición.

Idea: Como indexo en j y en el setAt se reemplaza la posición i+1, podríamos separar en casos, cuando $j = i+1$ y cuando $j \neq i+1$

$$\equiv -1 \leq i < |L|-1 \wedge_L ((\forall j: \mathbb{Z})(0 \leq j < |L| \wedge j = i+1 \rightarrow_L (\text{setAt}(L, i+1, a)[j] > 0 \wedge \text{setAt}(L, i+1, a)[j] \bmod 2 = 0)) \wedge (\forall j: \mathbb{Z})(0 \leq j < |L| \wedge j \neq i+1 \rightarrow_L (\text{setAt}(L, i+1, a)[j] > 0 \wedge \text{setAt}(L, i+1, a)[j] \bmod 2 = 0)))$$

Obs (V)

$$\equiv -1 \leq i < |L|-1 \wedge_L ((\text{setAt}(L, i+1, a)[i+1] > 0 \wedge \text{setAt}(L, i+1, a)[i+1] \bmod 2 = 0) \wedge (\forall j: \mathbb{Z})(0 \leq j < |L| \wedge j \neq i+1 \rightarrow_L (\text{setAt}(L, i+1, a)[j] > 0 \wedge \text{setAt}(L, i+1, a)[j] \bmod 2 = 0)))$$

Obs (IV)

$$\equiv -1 \leq i < |L|-1 \wedge_L (a > 0 \wedge a \bmod 2 = 0 \wedge (\forall j: \mathbb{Z})(0 \leq j < |L| \wedge j \neq i+1 \rightarrow_L (\text{setAt}(L, i+1, a)[j] > 0 \wedge \text{setAt}(L, i+1, a)[j] \bmod 2 = 0)))$$

Obs (III)

$$\equiv -1 \leq i < |L|-1 \wedge (a > 0 \wedge a \bmod 2 = 0 \wedge (\forall j: \mathbb{Z})(0 \leq j < |L| \wedge j \neq i+1 \rightarrow (L[j] > 0 \wedge L[j] \bmod 2 = 0)))$$

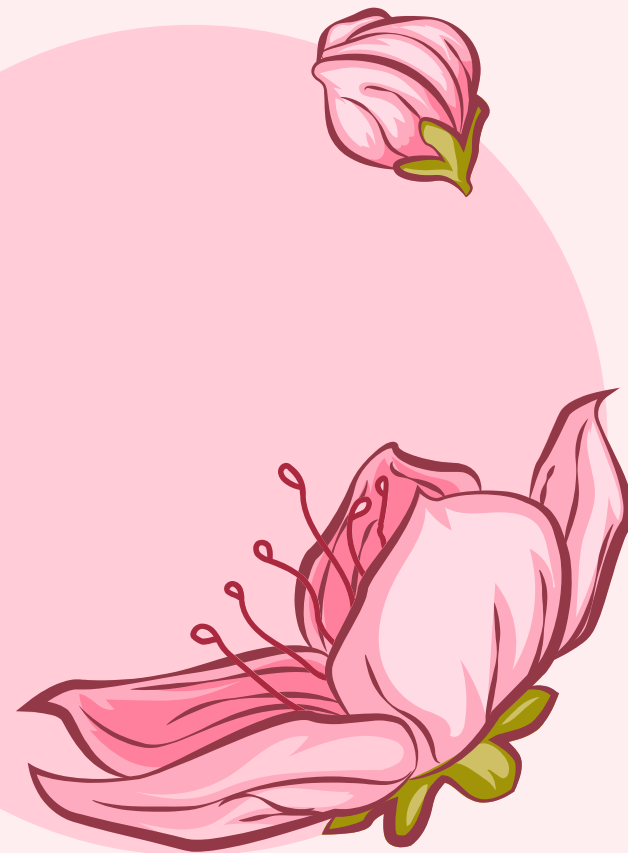
$$\equiv -1 \leq i < |L|-1 \wedge (\forall j: \mathbb{Z})((0 \leq j < |L| \wedge j \neq i+1) \rightarrow (L[j] > 0 \wedge L[j] \bmod 2 = 0)) \wedge a > 0 \wedge a \bmod 2 = 0$$

Que es lo mismo que habíamos dicho en α)



Intervalo

Volvemos en...



$S \equiv \text{if}(a \neq 19) \text{ then}$

$a := b$

else

$a := c+1$

endif

(5) $\text{wp}(S, a \geq 0)$

a)...

b)...



$S \equiv \text{if}(a \neq 19) \text{ then}$

$a := b$

else

$a := c+1$

endif

(5) $\text{wp}(S, a \geq 0)$

¿Cómo razono el a)?

Separo en casos: Si “a” es distinto de 19 se le asigna el valor de “b”, por lo que en ese caso “b” debería ser mayor o igual a 0 para cumplir Q. En caso de que “a” sea igual a 19, “c+1” es quien debe ser mayor o igual a 0 para cumplir Q.

a) En caso de que “a” sea distinto de 19, “b” debe ser mayor o igual a 0.

En caso de que “a” sea 19, “c” debe ser mayor o igual a -1

b) $wp(S, a \geq 0)$

$$Ax(4) \equiv \text{def}(a \neq 19) \wedge_L ((a \neq 19 \wedge wp(a:=b, a \geq 0)) \vee \\ \neg(a \neq 19) \wedge wp(a:=c+1, a \geq 0)))$$

$$Ax(1) \equiv \text{True} \wedge_L ((a \neq 19 \wedge (\text{def}(b) \wedge_L b \geq 0)) \vee \\ (a = 19 \wedge (\text{def}(c+1) \wedge_L c+1 \geq 0))) \\ \equiv (a \neq 19 \wedge b \geq 0) \vee (a = 19 \wedge c+1 \geq 0) \\ \equiv (a \neq 19 \wedge b \geq 0) \vee (a = 19 \wedge c \geq -1)$$

Que es lo mismo que habíamos dicho en a)

```
S ≡ if(a != 19) then
    a := b
else
    a := c+1
endif
```



Ejercicio:

Dada la siguiente especificación:

Proc problema(in $a: \mathbb{Z}$, in $x: \mathbb{R}$) {
Requiere: $\{a \geq 10 \vee (a < 10 \wedge x > -1)\}$

Asegura: $\{x > 4\}$
}

- a) Verificar formalmente si el programa S' cumple con la especificación
- b) ¿Y si el Requiere fuese $a < 5 \wedge x > 200$?

$S' \equiv$ if($a \geq 10$) then
 $x := 3$
else
 $x := x + 5$
endif

En otras palabras...

```
S' ≡ if(a ≥ 10) then  
      x := 3  
    else  
      x := x+5  
    endif
```

a) Verificar si la tripla de Hoare:

$\{a \geq 10 \vee (a < 10 \wedge x > -1)\} S' \{x > 4\}$ es valida

b) Verificar si la tripla de Hoare:

$\{a < 5 \wedge x > 200\} S' \{x > 4\}$ es valida

¿Cómo lo hacemos?

$(\{P\} S \{Q\} \text{ es valida}) \leftrightarrow (P \rightarrow WP(S, Q))$

$S' \equiv \text{if}(a \geq 10) \text{ then}$

$x := 3$

else

$x := x + 5$

endif

Paso 1:

Calcular la WP formalmente

$wp(S', x > 4)$

¿Cómo creen que se quedaría?

Notamos que si "a" fuese mayor/igual a 10, jamás se cumpliría "Q", ya que se le asignaría el valor 3 a "x". Entonces "a" debe ser si o si menor que 10. Luego, la única manera de que "x" sea mayor que 4 luego de ser aumentado consigo mismo por 5, es que en primera instancia haya sido mayor que -1.

Finalmente la WP debería quedar de la siguiente manera:

"a" debe ser menor que 10, y "x" debe ser mayor que -1

Calculo formal

$wp(S', x > 4)$

$$Ax(4) \equiv \text{def}(a \geq 10) \wedge_L ((a \geq 10 \wedge wp(x:=3, x>4)) \vee \\ \neg(a \geq 10) \wedge wp(x:=x+5, x>4)))$$

$$\begin{aligned} Ax(1) &\equiv \text{True} \wedge_L ((a \geq 10 \wedge (\text{def}(3) \wedge_L 3 > 4)) \vee \\ &\quad (a < 10 \wedge \text{def}(x+5) \wedge_L x+5 > 4)) \\ &\equiv (a \geq 10 \wedge \text{True} \wedge_L \text{False}) \vee (a < 10 \wedge \text{True} \wedge_L x > -1) \\ &\equiv \text{False} \vee (a < 10 \wedge x > -1) \\ &\equiv a < 10 \wedge x > -1 \end{aligned}$$

$S' \equiv$ if($a \geq 10$) then
 $x := 3$
else
 $x := x+5$
endif

Paso 2:

Verificar las implicaciones

a) ¿ $(a \geq 10 \vee (a < 10 \wedge x > -1)) \rightarrow (a < 10 \wedge x > -1)$?

No, porque puede ocurrir que el antecedente sea verdadero porque $a \geq 10$, y en ese caso el consecuente es falso.

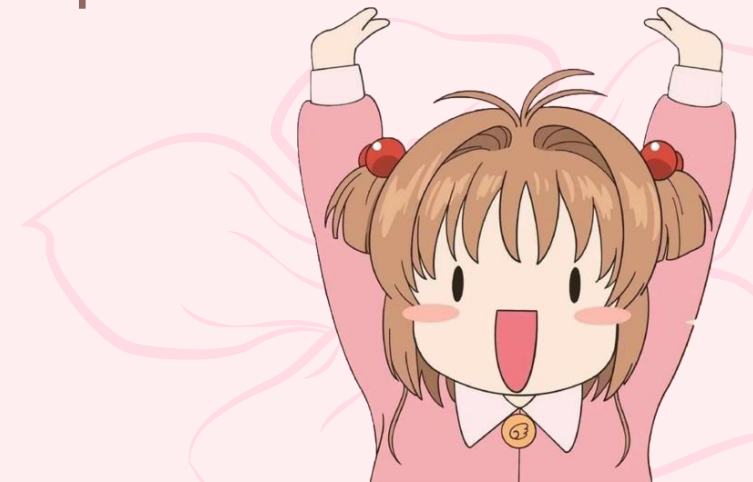
b) ¿ $(a < 5 \wedge x > 200) \rightarrow (a < 10 \wedge x > -1)$?

Si, porque siempre que a sea menor que 5, necesariamente es menor que 10, y también siempre que x sea mayor a 200 necesariamente es mayor que -1 .

Conclusiones

Luego de derivar la weakest precondition formalmente, y verificar las implicaciones, podemos asegurar que:

- a) El programa S no cumple con dicha especificación.
- b) Con dicho cambio el programa S si cumple con la especificación.



Consultas

